Scripts en Bash: ejercicios resueltos

Repaso para el examen de la UD 1

Alumno: Guillermo Martín

Curso: 2º ASIR

1. Ejercicio 1: notas.sh

Enunciado

Crea un script llamado `notas.sh` que gestione las notas de los alumnos de una clase. El script recibe exactamente 1 parámetro, el nombre del fichero donde se guardan las notas. Si el número de parámetros no es correcto, muestra el uso y termina. Si el fichero no existe, lo crea vacío y muestra "Se ha creado el fichero X".

El script enseña un menú una y otra vez hasta que el usuario elija salir. Cada opción va en su propia función:

1. Añadir alumno: pide el nombre, que no puede estar vacío, y la nota, que tiene que ser un número entero entre 0 y 10. Mientras la nota no sea válida, la vuelve a pedir. Guarda una línea `nombre:nota` en el fichero.
2. Listar alumnos: muestra todas las líneas numeradas. Si el fichero está vacío, muestra "No hay alumnos".
3. Estadísticas: total de alumnos, nota media (entera), número de aprobados (nota mayor o igual que 5) y de suspensos.
4. Buscar alumno: pide un nombre y muestra su nota, o "Alumno no encontrado".
5. Salir: pregunta "¿Seguro que quieres salir? (s/n)". Con s o S termina; con cualquier otra cosa vuelve al menú.

Si se elige una opción que no existe, muestra un error y vuelve al menú. Cada vez que se añade un alumno, se escribe en `registro.log` una línea con la fecha y hora, el usuario que ejecuta el script y el alumno añadido.

Pruebas

- `./notas.sh` y `./notas.sh a b` tienen que mostrar el uso.
- Al añadir un alumno con nota `abc`, luego `15` y luego `7`, la nota se vuelve a pedir hasta el 7.
- Con 3 o 4 alumnos, el listado y las estadísticas salen bien.
- Buscar un alumno que existe y otro que no.
- La opción `9` da error; salir con `n` vuelve al menú y con `s` termina.

Solución

```
#!/bin/bash
#
# Nombre: notas.sh
# Descripción: Gestiona las notas de los alumnos de una clase mediante un menú
# Uso: ./notas.sh <fichero_notas>
# Autor: Guillermo Martín
# Fecha: 2026-11-12
#


### VALIDACIÓN ###

# Exactamente 1 parámetro -> si no, mostrar el uso y salir
if [ $# -ne 1 ]
then
    echo "Uso: $0 <fichero_notas>"
    exit 1
fi

fichero="$1"

# -f = existe y es un fichero. Si NO existe, se crea vacío
if [ ! -f "$fichero" ]
then
    touch "$fichero"
    echo "Se ha creado el fichero $fichero"
fi


### FUNCIONES ###

mostrar_menu() {
    echo ""
    echo "===== GESTIÓN DE NOTAS ====="
    echo "1) Añadir alumno"
    echo "2) Listar alumnos"
    echo "3) Estadísticas"
    echo "4) Buscar alumno"
    echo "5) Salir"
}

# --- Opción 1 ---
anadir_alumno() {
    read -p "Nombre del alumno: " nombre

    # -z = vacío -> aviso y return (vuelve al menú, NO cierra el script)
    if [ -z "$nombre" ]
    then
        echo "ERROR: el nombre no puede estar vacío"
        return
    fi
```

```
    # Mientras NO sea un número O sea mayor que 10 -> volver a pedirla
    read -p "Nota (0-10): " nota
    while [[ ! "$nota" =~ ^[0-9]+$ ]] || [ "$nota" -gt 10 ]
    do
        echo "ERROR: la nota tiene que ser un número entre 0 y 10"
        read -p "Nota (0-10): " nota
    done

    # >> añade al final (con > se borraría todo)
    echo "$nombre:$nota" >> "$fichero"
    echo "Alumno $nombre añadido con un $nota"

    # Registro: fecha y hora - usuario - lo añadido
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $(whoami) - añadido $nombre:$nota" >>
registro.log
}

# --- Opción 2 ---
listar_alumnos() {
    # wc -l < fichero -> solo el número de líneas
    total=$(wc -l < "$fichero")

    if [ "$total" -eq 0 ]
    then
        echo "No hay alumnos"
        return
    fi

    # Leer el fichero línea a línea con un contador delante
    i=1
    while read -r linea
    do
        echo "$i) $linea"
        i=$((i + 1))
    done < "$fichero"
}

# --- Opción 3 ---
estadisticas() {
    total=$(wc -l < "$fichero")

    if [ "$total" -eq 0 ]
    then
        echo "No hay alumnos"
        return
    fi

    suma=0
    aprobados=0
    suspensos=0

    # cut -d: -f2 -> de "Ana:7" se queda con el 7
    for nota in $(cut -d: -f2 "$fichero")
    do
```

```
        suma=$((suma + nota))

        if [ "$nota" -ge 5 ]
        then
            aprobados=$((aprobados + 1))
        else
            suspensos=$((suspensos + 1))
        fi
    done

    media=$((suma / total))

    echo "Total de alumnos: $total"
    echo "Nota media:       $media"
    echo "Aprobados:        $aprobados"
    echo "Suspensos:        $suspensos"
}

# --- Opción 4 ---
buscar_alumno() {
    read -p "Nombre a buscar: " nombre

    # ^nombre: -> la línea EMPIEZA por ese nombre seguido de :
    linea=$(grep "^$nombre:" "$fichero")

    if [ -z "$linea" ]
    then
        echo "Alumno no encontrado"
    else
        nota=$(echo "$linea" | cut -d: -f2)
        echo "$nombre tiene un $nota"
    fi
}

# --- Opción 5 ---
salir() {
    read -p "¿Seguro que quieres salir? (s/n): " respuesta

    case "$respuesta" in
        s|S)
            echo "Saliendo..."
            exit 0
            ;;
        *)
            echo "Volviendo al menú"
            ;;
    esac
}


### SCRIPT ###

# Bucle infinito: menú -> leer opción -> el case llama a la función.
# Solo se sale con la opción 5 (exit 0 dentro de salir)
```

```
while true
do
    mostrar_menu
    read -p "Elige una opción [1-5]: " opcion

    case "$opcion" in
        1) anadir_alumno ;;
        2) listar_alumnos ;;
        3) estadisticas ;;
        4) buscar_alumno ;;
        5) salir ;;
        *) echo "Opción no válida, elige del 1 al 5" ;;
    esac
done
```

2. Ejercicio 2: usuarios.sh

Enunciado

Crea un script llamado `usuarios.sh` que gestione los usuarios de un grupo. Solo lo puede ejecutar root. Recibe exactamente 1 parámetro, el nombre de un grupo; si no, muestra el uso y termina. Si el grupo no existe, lo crea y muestra "Se ha creado el grupo X".

El menú se repite hasta salir, con una función por opción:

1. Crear usuario: pide el nombre y, si está vacío o ya existe, lo vuelve a pedir hasta que sea válido. Lo crea con el directorio personal en `/home/<grupo>/<nombre>`, el intérprete `/bin/bash`, la contraseña `<nombre>123`, y lo añade al grupo como grupo secundario.
2. Listar miembros del grupo: muestra los usuarios del grupo y cuántos son. Si no hay ninguno, muestra "El grupo no tiene miembros".
3. Eliminar usuario: pide el nombre y comprueba que existe. Pregunta "¿Seguro que quieres eliminar a X? (s/n)". Con s o S lo borra junto con su directorio personal; con cualquier otra respuesta, cancela.
4. Bloquear o desbloquear: pide el nombre, comprueba que existe y pregunta "¿Bloquear o desbloquear? (b/d)". Bloquea con `usermod -L` o desbloquea con `usermod -U`. Con otra letra, error.
5. Salir: antes de terminar, muestra cuántos usuarios normales hay en el sistema (UID mayor o igual que 1000).

Cada creación, eliminación, bloqueo o desbloqueo se apunta en `usuarios.log` con la fecha y hora, el usuario que ejecuta el script y la acción.

Pruebas

- Sin sudo: error de root. Sin parámetro o con dos: uso.
- Crear con nombre vacío y con `root`: lo vuelve a pedir. Después crear `ana` y `luis`.

- Listar: salen `ana` y `luis`, total 2.
- Bloquear a `ana` y comprobarlo con `sudo passwd -S ana` (sale una L).
- Eliminar a `luis` respondiendo `n` y después `s`.
- `cat usuarios.log` muestra cada acción.

Solución

```
#!/bin/bash   # shebang: este script lo ejecuta Bash (SIEMPRE la 1ª línea)
#
# Nombre: usuarios.sh
# Descripción: Gestiona los usuarios de un grupo mediante un menú
# Uso: sudo ./usuarios.sh <grupo>
# Autor: Guillermo Martín
# Fecha: 2026-11-12
#


### VALIDACIÓN ###

# 1) Solo root
if [ "$(whoami)" != "root" ]   # whoami = usuario que ejecuta. Si NO es root...
then
    echo "ERROR: debes ser administrador para ejecutar el script (usa sudo)"
    exit 1   # ...termina el script con error (1)
fi

# 2) Exactamente 1 parámetro
if [ $# -ne 1 ]   # $# = nº de parámetros. -ne = distinto de
then
    echo "Uso: $0 <grupo>"   # $0 = nombre del script
    exit 1
fi

grupo="$1"   # guardo el parámetro 1 con un nombre que se entienda

# 3) Si el grupo no existe, se crea
groupadd "$grupo" 2>/dev/null   # intenta crearlo. Si ya existe falla, y
2>/dev/null esconde el error
if [ $? -eq 0 ]   # $? = cómo acabó el último comando. 0 = bien -> se ha creado
then
    echo "Se ha creado el grupo $grupo"
fi   # si ya existía no se dice nada y el script sigue

log="usuarios.log"   # variable con el nombre del fichero de registro


### FUNCIONES ###

registrar() {   # Llamada: registrar "texto"
    echo "$(date '+%Y-%m-%d %H:%M:%S') - $(whoami) - $1" >> "$log"   # fecha -
usuario - texto, AÑADIDO (>>) al log
}   # $1 aquí es el texto que se le pasa a la función

mostrar_menu() {   # solo pinta las opciones
    echo ""   # línea en blanco para que no salga pegado
    echo "===== GESTIÓN DEL GRUPO $grupo ====="
    echo "1) Crear usuario"
    echo "2) Listar miembros del grupo"
    echo "3) Eliminar usuario"
```

```
    echo "4) Bloquear / desbloquear usuario"
    echo "5) Salir"
}

# --- Opción 1 ---
crear_usuario() {
    read -p "Nombre del nuevo usuario: " nombre   # pide el nombre y lo guarda
en $nombre

    while [ -z "$nombre" ] || id "$nombre" &>/dev/null   # MIENTRAS esté vacío
(-z) O ya exista (id va bien)...
    do
        if [ -z "$nombre" ]   # miro cuál de los dos fallos es para dar el
mensaje correcto
        then
            echo "ERROR: no has escrito ningún nombre"
        else
            echo "ERROR: el usuario $nombre ya existe"
        fi
        read -p "Nombre del nuevo usuario: " nombre   # ...lo vuelvo a pedir
(sin esto, bucle infinito)
    done   # cuando el nombre es válido, sale del while y sigue abajo

    mkdir -p "/home/$grupo"   # la carpeta de arriba tiene que existir. -p: no
falla si ya está
    useradd -m -d "/home/$grupo/$nombre" -s /bin/bash -G "$grupo" "$nombre"
    #       -m crea el home  -d dónde  -s shell   -G grupo secundario   nombre
(siempre al final)

    echo "$nombre:${nombre}123" | chpasswd   # pone la contraseña "nombre123"
sin preguntar (formato usuario:clave)
    # ${nombre}123 CON llaves: si no, Bash busca la variable "nombre123"

    echo "Usuario $nombre creado (home /home/$grupo/$nombre, contraseña
${nombre}123)"
    registrar "creado $nombre"   # lo apunto en el log
}

# --- Opción 2 ---
listar_miembros() {
    miembros=$(grep "$grupo:x:" /etc/group | cut -d: -f4)
    # grep busca la línea del grupo en /etc/group:  profes:x:1001:ana,luis
    # cut -d: -f4 la corta por los ":" y se queda con el trozo 4 (los miembros)
-> ana,luis
    # el :x: detrás del nombre es para que "profes" no encuentre también
"profesores"

    if [ -z "$miembros" ]   # si no hay nada -> el grupo está vacío
    then
        echo "El grupo no tiene miembros"
        return   # sale de la FUNCIÓN y vuelve al menú (exit cerraría el script)
    fi

    echo "Miembros del grupo $grupo:"
```

```
    total=0   # contador, empieza en 0
    for usuario in $(echo "$miembros" | tr ',' ' ')   # tr cambia las comas por
espacios: "ana luis" (el for separa por espacios)
    do
        echo "  - $usuario"   # muestra cada usuario
        total=$((total + 1))   # suma 1 al contador en cada vuelta
    done
    echo "Total: $total"
}

# --- Opción 3 ---
eliminar_usuario() {
    read -p "Nombre del usuario a eliminar: " nombre

    if ! id "$nombre" &>/dev/null   # ! = "si NO". Si el usuario NO existe...
    then
        echo "ERROR: el usuario $nombre no existe"
        return   # ...error y vuelta al menú
    fi

    read -p "¿Seguro que quieres eliminar a $nombre? (s/n): " respuesta
    case "$respuesta" in   # según lo que conteste:
        s|S)   # s o S (| = "o")
            userdel -r "$nombre" 2>/dev/null   # borra el usuario y su home (-
r). 2>/dev/null esconde avisos
            echo "Usuario $nombre eliminado"
            registrar "eliminado $nombre"
            ;;   # ;; cierra cada opción del case
        *)   # cualquier otra cosa
            echo "Cancelado"
            ;;
    esac   # esac cierra el case ("case" al revés)
}

# --- Opción 4 ---
bloquear_usuario() {
    read -p "Nombre del usuario: " nombre

    if ! id "$nombre" &>/dev/null   # igual que en eliminar: si NO existe, error
y al menú
    then
        echo "ERROR: el usuario $nombre no existe"
        return
    fi

    read -p "¿Bloquear o desbloquear? (b/d): " accion
    case "$accion" in
        b|B)
            usermod -L "$nombre"   # -L = Lock: bloquea la cuenta (no puede
iniciar sesión)
            echo "Usuario $nombre bloqueado"
            registrar "bloqueado $nombre"
            ;;
        d|D)
```

```
            usermod -U "$nombre"   # -U = Unlock: la desbloquea
            echo "Usuario $nombre desbloqueado"
            registrar "desbloqueado $nombre"
            ;;
        *)
            echo "ERROR: tienes que escribir b o d"
            ;;
    esac
    # Para comprobarlo en la terminal: sudo passwd -S nombre  -> L = bloqueado,
P = normal
}

# --- Opción 5 ---
salir() {
    normales=0   # contador de usuarios normales
    for uid in $(cut -d: -f3 /etc/passwd)   # campo 3 de /etc/passwd = UID de
cada usuario (0, 1, 2... 1000...)
    do
        if [ "$uid" -ge 1000 ] && [ "$uid" -ne 65534 ]   # -ge = mayor o igual.
65534 es "nobody", no es una persona
        then
            normales=$((normales + 1))   # es un usuario normal -> sumo 1
        fi
    done

    echo "Usuarios normales en el sistema: $normales"
    echo "Saliendo..."
    exit 0   # cierra el script (bien). Es la única salida del while true
}


### SCRIPT ###

while true   # bucle infinito: se repite hasta el exit de salir()
do
    mostrar_menu   # pinta el menú (se llama a la función por su nombre, sin
paréntesis)
    read -p "Elige una opción [1-5]: " opcion   # guarda lo que escribe en
$opcion

    case "$opcion" in   # según la opción, llama a una función
        1) crear_usuario ;;
        2) listar_miembros ;;
        3) eliminar_usuario ;;
        4) bloquear_usuario ;;
        5) salir ;;
        *) echo "Opción no válida, elige del 1 al 5" ;;   # cualquier otra cosa
-> aviso y vuelve el menú
    esac
done   # vuelve arriba: menú otra vez
```

3. Ejercicio 3: numeros.sh

Enunciado

El script recibe 1 parámetro N, que tiene que ser un número entre 1 y 20. Muestra los números del 1 al N diciendo si cada uno es par o impar. Al final muestra la suma total y cuántos pares e impares ha habido. Hay que usar una función `es_par`.

Pruebas

- `./numeros.sh`, `./numeros.sh abc` y `./numeros.sh 30` dan uso o error.
- `./numeros.sh 5`: suma 15, pares 2, impares 3.

Solución

```
#!/bin/bash
#
# Nombre: numeros.sh
# Descripción: Muestra los números del 1 al N diciendo si son pares o impares,
#              y al final la suma y cuántos pares e impares hay.
# Uso: ./numeros.sh <N>        (N entre 1 y 20)
# Autor: Guillermo Martín
#


### VALIDACIÓN ###

if [ $# -ne 1 ]   # tiene que haber exactamente 1 parámetro
then
    echo "Uso: $0 <N>   (N entre 1 y 20)"
    exit 1
fi

if [[ ! "$1" =~ ^[0-9]+$ ]]   # tiene que ser un número
then
    echo "ERROR: '$1' no es un número"
    exit 1
fi

if [ "$1" -lt 1 ] || [ "$1" -gt 20 ]   # y estar entre 1 y 20 (esto SIEMPRE
después de la regex)
then
    echo "ERROR: el número tiene que estar entre 1 y 20"
    exit 1
fi

n="$1"   # guardo el parámetro con un nombre claro


### FUNCIONES ###

es_par() {   # devuelve "verdad" si $1 es par -> se usa en un if
    [ $(( $1 % 2 )) -eq 0 ]   # % = resto de dividir. Resto 0 entre 2 -> par
}


### SCRIPT ###

suma=0   # contadores a 0 antes del bucle
pares=0
impares=0

for i in $(seq 1 "$n")   # de 1 a N (con variable se usa seq, no {1..$n})
do
    if es_par "$i"   # llamo a la función con el número de esta vuelta
    then
        echo "$i es par"
```

```
        pares=$((pares + 1))
    else
        echo "$i es impar"
        impares=$((impares + 1))
    fi
    suma=$((suma + i))   # voy sumando todos
done

echo "-----------------"
echo "Suma total: $suma"
echo "Pares: $pares   Impares: $impares"
```

4. Ejercicio 4: fichero.sh

Enunciado

El script recibe 1 parámetro, que tiene que ser un fichero que exista. Muestra un menú que se repite hasta salir:

1. Ver el contenido.
2. Contar líneas y palabras.
3. Añadir una línea. Si se deja vacía, avisa y no hace nada.
4. Hacer una copia con la fecha en el nombre.
5. Salir.

Pruebas

- `./fichero.sh /no/existe` da error.
- Añadir una línea vacía avisa; añadir una con texto la pone al final.
- La copia aparece como `fichero_AAAA-MM-DD`.

Solución

```
#!/bin/bash
#
# Nombre: fichero.sh
# Descripción: Menú para trabajar con un fichero de texto que se pasa por
parámetro.
# Uso: ./fichero.sh <fichero>
# Autor: Guillermo Martín
#


### VALIDACIÓN ###

if [ $# -ne 1 ]   # exactamente 1 parámetro
then
    echo "Uso: $0 <fichero>"
    exit 1
fi

fichero="$1"

if [ ! -f "$fichero" ]   # -f = es un fichero. ! = si NO lo es
then
    echo "ERROR: '$fichero' no existe o no es un fichero"
    exit 1
fi


### FUNCIONES ###

mostrar_menu() {
    echo ""
    echo "=== FICHERO: $fichero ==="
    echo "1) Ver contenido"
    echo "2) Contar líneas y palabras"
    echo "3) Añadir una línea"
    echo "4) Hacer copia con la fecha"
    echo "5) Salir"
}

ver() {
    cat "$fichero"   # muestra todo el fichero
}

contar() {
    echo "Líneas:   $(wc -l < "$fichero")"   # wc -l = líneas. Con < sale solo
el número
    echo "Palabras: $(wc -w < "$fichero")"   # wc -w = palabras
}

anadir() {
    read -p "Texto a añadir: " texto
    if [ -z "$texto" ]   # -z = vacío
```

```
    then
        echo "No has escrito nada"
        return   # vuelve al menú sin hacer nada
    fi
    echo "$texto" >> "$fichero"   # >> AÑADE al final (> lo borraría todo)
    echo "Línea añadida"
}

copia() {
    copia="${fichero}_$(date +%Y-%m-%d)"   # notas.txt -> notas.txt_2026-11-12
    cp "$fichero" "$copia"   # copia el fichero
    echo "Copia creada: $copia"
}


### SCRIPT ###

while true   # se repite hasta elegir la 5
do
    mostrar_menu
    read -p "Opción: " opcion

    case "$opcion" in
        1) ver ;;
        2) contar ;;
        3) anadir ;;
        4) copia ;;
        5)
            echo "Adiós"
            exit 0   # única salida del bucle
            ;;
        *) echo "Opción no válida" ;;   # cualquier otra cosa
    esac
done
```

5. Ejercicio 5: backup_usuario.sh

Enunciado

Solo lo puede ejecutar root. Recibe 1 parámetro, un nombre de usuario que tiene que existir y tener su carpeta en `/home`. Hace una copia comprimida de su home en `/backup` (la carpeta se crea si no existe) con el nombre `usuario_AAAAMMDD_HHMMSS.tar.gz`. Al terminar muestra la copia creada y cuántas copias de ese usuario hay ya.

Pruebas

- Sin sudo, sin parámetro o con un usuario que no existe: error.
- Con un usuario real: aparece el `.tar.gz` en `/backup` y el número de copias.

Solución

```
#!/bin/bash
#
# Nombre: backup_usuario.sh
# Descripción: Hace una copia comprimida del directorio personal de un usuario
#              en /backup, con la fecha y la hora en el nombre.
# Uso: sudo ./backup_usuario.sh <usuario>
# Autor: Guillermo Martín
#


### VALIDACIÓN ###

if [ "$(whoami)" != "root" ]   # solo root (para poder leer el home de
cualquiera)
then
    echo "ERROR: debes ser administrador (usa sudo)"
    exit 1
fi

if [ $# -ne 1 ]   # exactamente 1 parámetro
then
    echo "Uso: $0 <usuario>"
    exit 1
fi

usuario="$1"

if ! id "$usuario" &>/dev/null   # el usuario tiene que existir (id falla si no
existe)
then
    echo "ERROR: el usuario $usuario no existe"
    exit 1
fi

if [ ! -d "/home/$usuario" ]   # y tener su carpeta en /home
then
    echo "ERROR: $usuario no tiene directorio en /home"
    exit 1
fi


### FUNCIONES ###

hacer_copia() {
    mkdir -p /backup   # crea /backup si no existe (-p: sin error si ya está)
    nombre="${usuario}_$(date +%Y%m%d_%H%M%S).tar.gz"   #
ana_20261112_183000.tar.gz
    tar -czf "/backup/$nombre" "/home/$usuario" 2>/dev/null
    # tar: -c crear  -z comprimir  -f nombre del fichero. 2>/dev/null esconde
avisos de tar
}
```

```
### SCRIPT ###

hacer_copia   # llamo a la función

if [ $? -eq 0 ]   # $? = cómo acabó lo último (la función). 0 = bien
then
    echo "Copia creada: /backup/$nombre"
    ls -lh "/backup/$nombre"   # -l detalles  -h tamaño legible (K, M)
else
    echo "ERROR: no se ha podido hacer la copia"
    exit 1
fi

total=$(ls /backup | grep "$usuario" | wc -l)   # cuántas copias de este usuario
hay en /backup
echo "Copias de $usuario guardadas: $total"
```

6. Ejercicio 6: prioridad.sh

Enunciado

Solo lo puede ejecutar root y no admite parámetros. Pide un PID y lo vuelve a pedir hasta que exista un proceso con ese PID. Muestra el nombre del proceso y su nice actual. Pide un nuevo nice, que tiene que ser un número entre -20 y 19, y lo vuelve a pedir hasta que sea válido. Cambia la prioridad, muestra el nice de antes y el de ahora, y lo apunta en `resumen.txt`. Al final pregunta si quieres ver `resumen.txt`.

Pruebas

- En otra terminal, `sleep 1000 &` para tener un PID con el que probar.
- PID `abc` o `999999`: lo vuelve a pedir.
- Nice `30` o `x`: lo vuelve a pedir. Con `5`, antes 0 y ahora 5.

Solución

```
#!/bin/bash
#
# Nombre: prioridad.sh
# Descripción: Cambia la prioridad (nice) de un proceso y lo apunta en
resumen.txt
# Uso: sudo ./prioridad.sh        (no necesita parámetros)
# Autor: Guillermo Martín
#


### VALIDACIÓN ###

if [ "$(whoami)" != "root" ]   # solo root (para poder subir la prioridad)
then
    echo "ERROR: debes ser administrador (usa sudo)"
    exit 1
fi

if [ $# -ne 0 ]   # no admite parámetros
then
    echo "Uso: $0"
    exit 1
fi


### FUNCIONES ###

ver_nice() {   # Llamada: ver_nice PID -> escribe su nice
    ps -p "$1" -o ni= | tr -d ' '   # ps -o ni= saca solo el nice. tr -d ' '
quita espacios
}


### SCRIPT ###

# 1) Pedir el PID hasta que exista
read -p "PID del proceso: " pid
while ! ps -p "$pid" &>/dev/null   # mientras NO exista un proceso con ese
PID...
do
    echo "ERROR: ese PID no existe"
    read -p "PID del proceso: " pid   # ...lo vuelvo a pedir
done

nombre=$(ps -p "$pid" -o comm=)   # nombre del proceso
antes=$(ver_nice "$pid")   # guardo el nice de ANTES
echo "Proceso: $nombre   Nice actual: $antes"

# 2) Pedir el nuevo nice hasta que sea un número entre -20 y 19
read -p "Nuevo nice (-20 a 19): " nuevo
while [[ ! "$nuevo" =~ ^-?[0-9]+$ ]] || [ "$nuevo" -lt -20 ] || [ "$nuevo" -gt
19 ]
```

```
do   # ^-? = puede llevar un - delante (números negativos)
    echo "ERROR: tiene que ser un número entre -20 y 19"
    read -p "Nuevo nice (-20 a 19): " nuevo
done

# 3) Cambiarlo y enseñar antes y después
renice -n "$nuevo" -p "$pid" > /dev/null   # > /dev/null: no enseñar el mensaje
de renice
despues=$(ver_nice "$pid")
echo "Antes: $antes   Ahora: $despues"

# 4) Guardarlo en resumen.txt
echo "$(date '+%Y-%m-%d %H:%M:%S') - PID $pid ($nombre) - nice $antes ->
$despues" >> resumen.txt

# 5) Preguntar si quiere ver el registro
read -p "¿Quieres ver resumen.txt? (s/n): " resp
case "$resp" in
    s|S) cat resumen.txt ;;
    *)   echo "Vale, adiós" ;;
esac
```

7. Ejercicio 7: impares.sh

Enunciado

El script recibe 1 parámetro N, que tiene que ser un número entre 1 y 50. Si no recibe exactamente 1 parámetro, muestra el uso; si N no es un número o no está en el rango, muestra un error. Recorre los números del 1 al N y muestra solo los impares, saltándose los pares con `continue`. Usa una función `es_impar`. Al final muestra cuántos impares ha habido y cuánto suman.

Pruebas

- `./impares.sh`, `./impares.sh hola` y `./impares.sh 60` dan uso o error.
- `./impares.sh 9`: 1 3 5 7 9, total 5, suma 25.

Solución

```
#!/bin/bash
#
# Nombre: impares.sh
# Descripción: Muestra solo los números impares del 1 al N, cuántos hay y cuánto
suman.
# Uso: ./impares.sh <N>        (N entre 1 y 50)
# Autor: Guillermo Martín
#


### VALIDACIÓN ###

if [ $# -ne 1 ]   # tiene que haber exactamente 1 parámetro
then
    echo "Uso: $0 <N>   (N entre 1 y 50)"
    exit 1
fi

if [[ ! "$1" =~ ^[0-9]+$ ]]   # tiene que ser un número
then
    echo "ERROR: '$1' no es un número"
    exit 1
fi

if [ "$1" -lt 1 ] || [ "$1" -gt 50 ]   # y estar entre 1 y 50 (siempre después
de la regex)
then
    echo "ERROR: el número tiene que estar entre 1 y 50"
    exit 1
fi

n="$1"   # guardo el parámetro con un nombre claro


### FUNCIONES ###

es_impar() {   # devuelve "verdad" si $1 es impar -> se usa en un if
    [ $(( $1 % 2 )) -ne 0 ]   # resto de dividir entre 2 distinto de 0 -> impar
}


### SCRIPT ###

total=0   # contadores a 0 antes del bucle
suma=0

for i in $(seq 1 "$n")   # de 1 a N
do
    if ! es_impar "$i"   # si NO es impar (es par)...
    then
        continue   # ...me lo salto y paso al siguiente número
    fi
```

```
    echo "$i"   # aquí solo llegan los impares
    total=$((total + 1))   # cuento uno más
    suma=$((suma + i))   # y lo sumo
done

echo "-----------------"
echo "Impares: $total"
echo "Suma: $suma"
```

Fallos típicos en este ejercicio:

- `[ $1 % 2 -ne 0 ]` no calcula nada. Las cuentas van siempre dentro de `$(( ))`.
- Dentro del `for`, el número que cambia es `$i`, no `$1`.
- `suma=$((suma + 1))` cuenta; para sumar es `suma=$((suma + i))`.
- El `continue` sirve antes del código que se quiere saltar, no al final del bucle.

8. Ejercicio 8: tabla.sh

Enunciado

El script recibe 2 parámetros: un número (entre 1 y 10) y un límite (entre 1 y 20). Si no son exactamente 2, muestra el uso. Los dos tienen que ser números y estar en su rango; si no, muestra un error. Muestra la tabla de multiplicar del número desde el 1 hasta el límite con el formato `7 x 3 = 21`. El cálculo lo hace una función `multiplicar` que devuelve el resultado con `echo`. Al final indica cuántos resultados son mayores que 50.

Pruebas

- `./tabla.sh 7`, `./tabla.sh 7 x` y `./tabla.sh 15 5` dan uso o error.
- `./tabla.sh 7 10`: la tabla del 7 hasta el 10 y 3 resultados mayores que 50.

Solución

```
#!/bin/bash
#
# Nombre: tabla.sh
# Descripción: Muestra la tabla de multiplicar de un número hasta un límite
# Uso: ./tabla.sh <numero> <limite>     (numero 1-10, limite 1-20)
# Autor: Guillermo Martín
#


### VALIDACIÓN ###

# 1) Exactamente 2 parámetros
if [ $# -ne 2 ]
then
    echo "Uso: $0 <numero> <limite>   (numero 1-10, limite 1-20)"
    exit 1
fi

num="$1"   # guardo cada parámetro con un nombre claro
limite="$2"

# 2) Los dos tienen que ser números
if [[ ! "$num" =~ ^[0-9]+$ ]] || [[ ! "$limite" =~ ^[0-9]+$ ]]
then
    echo "ERROR: los dos parámetros tienen que ser números"
    exit 1
fi

# 3) El número entre 1 y 10
if [ "$num" -lt 1 ] || [ "$num" -gt 10 ]
then
    echo "ERROR: el número tiene que estar entre 1 y 10"
    exit 1
fi

# 4) El límite entre 1 y 20
if [ "$limite" -lt 1 ] || [ "$limite" -gt 20 ]
then
    echo "ERROR: el límite tiene que estar entre 1 y 20"
    exit 1
fi


### FUNCIONES ###

multiplicar() {   # Llamada: multiplicar 7 3  -> escribe 21
    echo $(( $1 * $2 ))   # $1 y $2 aquí son los que recibe la FUNCIÓN
}


### SCRIPT ###
```

```
mayores=0   # contador de resultados mayores que 50

echo "Tabla del $num:"
for i in $(seq 1 "$limite")   # de 1 hasta el límite
do
    resultado=$(multiplicar "$num" "$i")   # recojo lo que "devuelve" la función
con $( )
    echo "$num x $i = $resultado"

    if [ "$resultado" -gt 50 ]   # -gt = mayor que
    then
        mayores=$((mayores + 1))   # uno más mayor que 50
    fi
done

echo "-----------------"
echo "Resultados mayores que 50: $mayores"
```

Si el enunciado no pide la función, la cuenta se puede hacer directamente dentro del bucle con `resultado=$(( num * i ))`.

9. Ejercicio 9: cuenta_atras.sh (pendiente)

Enunciado

El script recibe 0 o 1 parámetros. Si recibe más de 1, muestra el uso. Si recibe 1, ese es el número desde el que cuenta, y tiene que ser un número entre 1 y 10; si no, error. Si no recibe ninguno, pide el número con `read` y lo vuelve a pedir hasta que sea un número entre 1 y 10. Hace una cuenta atrás con un `while`, un número por segundo (`sleep 1`), y al final muestra "¡Despegue!".

Pruebas

- `./cuenta_atras.sh 1 2`: uso. `./cuenta_atras.sh 20`: error.
- `./cuenta_atras.sh 3`: 3, 2, 1, ¡Despegue!
- `./cuenta_atras.sh` escribiendo `abc`, `15` y `4`: lo pide dos veces más y cuenta desde 4.

10. Ejercicio 10: mayor_menor.sh (pendiente)

Enunciado

El script recibe al menos 2 parámetros, todos números enteros (pueden ser negativos). Si recibe menos de 2, muestra el uso. Recorre todos los parámetros con `for n in "$@"` y, si alguno no es un número, dice cuál es y termina. Muestra el mayor, el menor, la suma y la media entera. Usa una función `es_numero` que se pueda usar en un `if`.

Pruebas

- `./mayor_menor.sh 5`: uso. `./mayor_menor.sh 4 x 9`: error con `x`.
- `./mayor_menor.sh 4 -2 9 1`: mayor 9, menor -2, suma 12, media 3.

Pista: antes del `for`, guarda el primer parámetro como mayor y como menor. En cada vuelta, si el número supera a `mayor`, pasa a ser el nuevo mayor; con `menor` igual pero al revés.